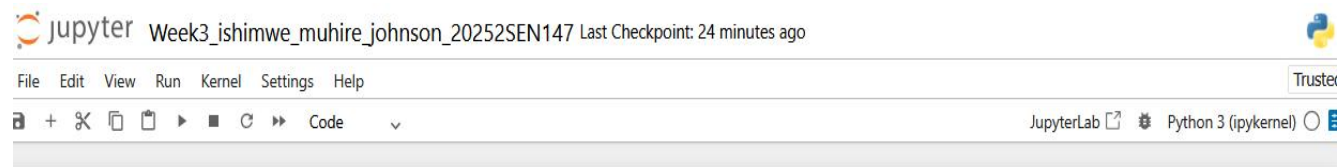


Screen Shoots



[1]: *## Exercises 1.1 Making Decisions*

```
scores = [91, 76, 64, 52, 43]

for score in scores:

    if score >= 85:
        print("A")

    elif score >= 70:
        print("B")

    elif score >= 60:
        print("C")

    elif score >= 50:
        print("D")

    else:
        print("F")
```

A
B
C
D
F

[2]: *## Exercises 1.2*

```
def validation (score, attendance):
    if score >=50 and attendance >=75:
        print("Validated")

    elif score>=50 and attendance <75 :
        print("Blocked: low attendance")

    else:
        print ("Failed")

validation (76, 92)
validation (76, 68)
validation (43, 95)
```

Validated
Blocked: low attendance
Failed

[3]: *## Exercises 1.3*

```
# A: Since elif stops at the FIRST condition that is True, a score of 76
#     would match `score >= 50` first and be graded D instead of B -
#     every score of 70 or higher (up to 84) would be misclassified as D,
#     because the Ladder never gets the chance to test the higher bands.
```

```
[4]: ## Exercises 2.1
scores = [72, 85, 91, 64, 78, 47, 88, 55, 93, 61]
for s in scores:
    if s >= 80:
        print(f"Top score: {s}")
```

Top score: 85
Top score: 91
Top score: 88
Top score: 93

```
[6]: # Exercise 2.2
total = 0
count = 0
passed = 0
failed = 0

for s in scores:
    total = total + s
    count = count + 1
    if s >= 50:
        passed = passed + 1
    else:
        failed = failed + 1

average = total / count # computed once, after the loop

print(f"Total : {total}")
print(f"Count : {count}")
print(f"Average : {average:.1f}")
print(f"Passed : {passed}")
print(f"Failed : {failed}")
```

Total : 734
Count : 10
Average : 73.4
Passed : 9
Failed : 1

```
[7]: # Exercises 2.3
target = 250000
monthly_saving = 15000
saved = 0
months = 0

while saved < target:
    saved = saved + monthly_saving
    months = months + 1
print(f"You need {months} months to save {saved} RWF (target was {target} RWF).")

You need 17 months to save 255000 RWF (target was 250000 RWF).
```

```
[19]: # Exercise 3.1
def get_grade(score):
    if score >= 85:
        return "A"
    elif score <= 84 and score >= 70:
        return "B"
    elif score <= 79 and score >= 60:
        return "C"
    elif score <= 59 and score >= 50:
        return "D"
    elif score < 50:
        return "F"

print(get_grade(91))
print(get_grade(76))
print(get_grade(64))
print(get_grade(52))
print(get_grade(43))
```

A
B
C
D
F

```
[21]: # exercise 3.2
def pass_rate(score_list):

    total_count = 0
    pass_count = 0
    for s in score_list:
        total_count += 1
        if s >= 50:
            pass_count += 1
    return (pass_count / total_count) * 100

scores = [72, 85, 91, 64, 78, 47, 88, 55, 93, 61]
print(f"pass_rate(scores) -> {pass_rate(scores)}")
```

pass_rate(scores) -> 90.0

```
[22]: # exercise 4.1

import pandas as pd
df = pd.read_excel("week3_students.xlsx")
df.to_csv("week3_students.csv", index=False)
```

[26]: # exercise 4.1

```
with open("week3_students.csv") as f:
    lines = f.readlines()

print(f"(a) Number of lines : {len(lines)}")
print(f"(b) Header line : {lines[0].strip()}")
print(f"(c) First student : {lines[1].strip()}")
```

(a) Number of lines : 41
 (b) Header line : student_id,name,age,gender,program,district,attendance_pct,score
 (c) First student : AUCA001,Aline Uwase,19,F,Information Systems,Gasabo,92,85

[27]: # exercise 4.2

```
names = []
for line in lines[1:]:
    parts = line.strip().split(",")
    names.append(parts[1])

print(f"Number of names : {len(names)}")
print(f"First 5 names : {names[:5]}")
```

Number of names : 40
 First 5 names : ['Aline Uwase', 'Eric Niyonzima', 'Diane Mukamana', 'Jean Bosco Habimana', 'Sandrine Ingabine']

[28]: # exercise 5.1

```
total = 0
valid_count = 0
bad_count = 0
bad_ids = []

for line in lines[1:]:
    parts = line.strip().split(",")
    student_id = parts[0]
    try:
        score = int(parts[7]) # this line raises ValueError if not numeric
        total += score
        valid_count += 1
    except ValueError:
        bad_count += 1
        bad_ids.append(student_id)

average_valid = total / valid_count

print(f"Valid scores : {valid_count}")
print(f"Bad scores : {bad_count}")
print(f"Bad IDs : {bad_ids}")
print(f"Average (valid only) : {average_valid:.1f}")
```

Valid scores : 37
 Bad scores : 3
 Bad IDs : ['AUCA008', 'AUCA020', 'AUCA032']
 Average (valid only) : 75.3

Average (valid only) : 75.3

```
[32]: # exercise 5.2
passed = 0
failed = 0
best_score = -1
best_name = ""

for line in lines[1:]:
    parts = line.strip().split(",")
    student_id = parts[0]
    name = parts[1]
    if student_id in bad_ids:
        continue
    score = int(parts[7])
    if score >= 50:
        passed += 1
    else:
        failed += 1
    if score > best_score:
        best_score = score
        best_name = name

print(f"Passed : {passed}")
print(f"Failed : {failed}")
print(f"Top student : {best_name} with score {best_score} (grade {get_grade(best_score)})")
```

Passed : 35
Failed : 2
Top student : Bonaventure Nkurunziza with score 95 (grade A)

```
[33]: # exercise 5.3
sums = {}
counts = {}

for line in lines[1:]:
    parts = line.strip().split(",")
    student_id = parts[0]
    if student_id in bad_ids:
        continue
    d = parts[5]
    s = int(parts[7])
    sums[d] = sums.get(d, 0) + s
    counts[d] = counts.get(d, 0) + 1

best_district = ""
best_district_avg = -1
for d in sums:
    avg = sums[d] / counts[d]
    print(f"{d:12s} -> average {avg:.1f} (n={counts[d]})")
    if avg > best_district_avg:
        best_district_avg = avg
        best_district = d
print(f"\nHighest-average district: {best_district} ({best_district_avg:.1f})")
```

Gasabo -> average 80.4 (n=5)
Musanze -> average 65.2 (n=5)



```
print(f"{d:12s} -> average {avg:.1f} (n={counts[d]})")
if avg > best_district_avg:
    best_district_avg = avg
    best_district = d
print(f"\nHighest-average district: {best_district} ({best_district_avg:.1f})")
```

```
Gasabo    -> average 80.4 (n=5)
Musanze   -> average 65.2 (n=5)
Muhanga   -> average 87.0 (n=5)
Kicukiro  -> average 64.0 (n=4)
Huye      -> average 68.4 (n=5)
Nyagatare -> average 82.8 (n=5)
Nyarugenge -> average 74.8 (n=5)
Rubavu    -> average 78.7 (n=3)
```

Highest-average district: Muhanga (87.0)

```
[34]: # bonus
report_lines = [
    f"Week 3 Report\n",
    f"Total students : {len(lines) - 1}\n",
    f"Valid scores   : {valid_count}\n",
    f"Bad scores     : {bad_count}\n",
    f"Bad IDs        : {bad_ids}\n",
    f"Average (valid) : {average_valid:.1f}\n",
    f"Top student    : {best_name} ({best_score}, grade {get_grade(best_score)})\n",
    f"Best district   : {best_district} ({best_district_avg:.1f})\n",
]

with open("week3_report.txt", "w") as f:
    f.writelines(report_lines)
```

```
[36]: # Read it back and print it, to prove the write worked
```

```
with open("week3_report.txt") as f:
    print(f.read())
```

```
Week 3 Report
Total students : 40
Valid scores   : 37
Bad scores     : 3
Bad IDs        : ['AUCA008', 'AUCA020', 'AUCA032']
Average (valid) : 75.3
Top student    : Bonaventure Nkurunziza (95, grade A)
Best district   : Muhanga (87.0)
```

```
[ ]: # Reflection
#Once bad records passed ~10-15% I would stop silently skipping and instead flag the file for manual review, or try to
#recover values from another source rather than just dropping them,ince dropping that much data could bias the average.
#---- what i had to do----
#Log every bad row with its reason for failing, rather than just counting it, so the pattern is
#check whether the broken values cluster around a particular column, source file, or
#time period, since that usually points to how the corruption happened; and
#go back to the original data source (or whoever entered the data) to recover the true values rather than quietly treating them as missing.
```